



LA COUCHE DAL

Data Access Layer



DATA ACCESS LAYER

- Motivations :
 - La plupart des applications ont **besoin de gérer des données persistantes**
 - La persistance des données peut être gérée par :
 - **Des systèmes différents selon la configuration** de déploiement
 - **Des systèmes évoluant dans le temps**
- Principe :
 - **Isoler la couche de persistance** du reste de l'application
- Intérêt :
 - La **couche métier** devient indépendante de la persistance



DATA ACCESS LAYER

Encapsuler le code lié à la persistance des données dans des objets

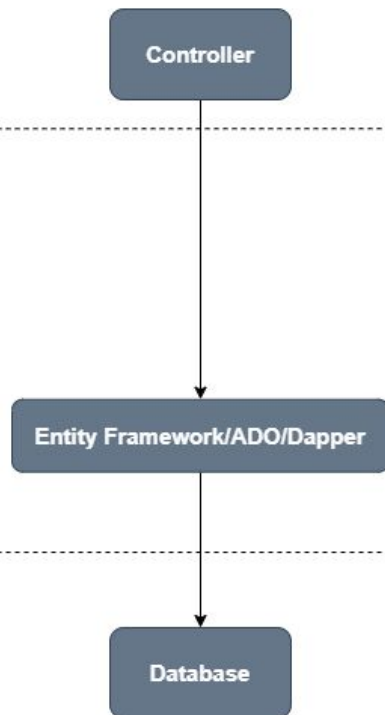
La DAL utilise le pattern REPOSITORY, pour manipuler les BO(Business Object).

Repository:

- Il existe **un unique Repository pour chaque BO**
- L'objet Repository **fait appel à un ORM ou un Driver pour persister les BO**
- Il permet **de changer rapidement de SGBD en créant une nouvelle implémentation**
- Il contient au minimum les méthodes:
 - **GET**: Permet de récupérer toutes les entités
 - **ADD**: Permet d'ajouter une entité
 - **FindByID**: Permet de récupérer une entité en fonction de son identifiant unique
 - **Delete**: Permet de supprimer une entité en fonction de son identifiant unique
 - **Modify**: Permet de modifier une entité en fonction de son identifiant unique

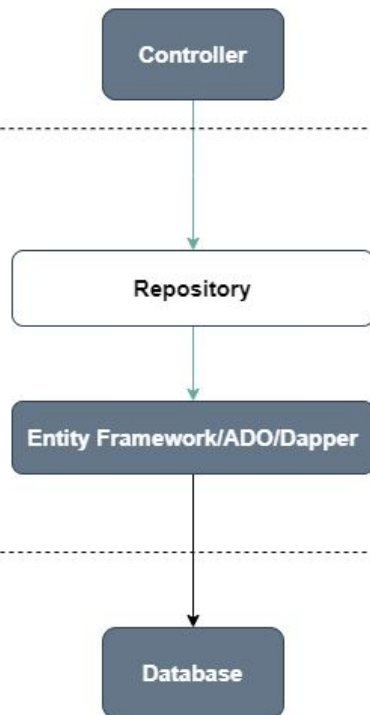
No Repository

The data will be accessed directly from the DbContext at the controller



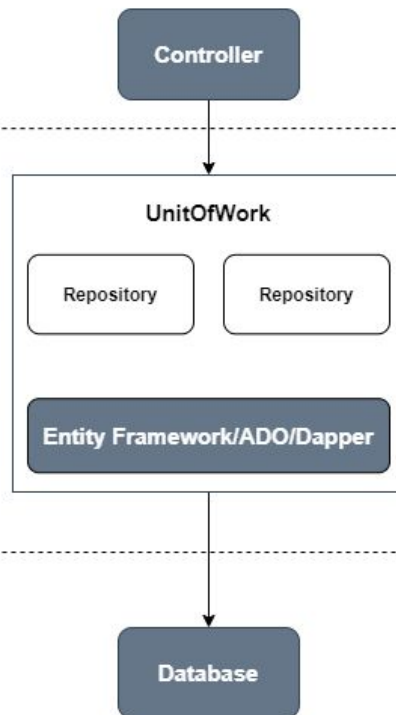
Repository

Data will be accessed via repository and logic will be hidden from the application layer



Repository + UnitOfWork

All the transactions of Repositories will be one single transaction



DAL

PATTERN : UNITOFWORK

- Motivation :
 - Avoir un système de transaction commun à tous nos repository
- Principe :
 - Isoler les repository du reste de l'application
- Intérêt :
 - L'accès aux repositories est centralisée dans le UOW